

# Inventory Management System

Project 05 | Python | JSON-based persistent storage

---

## Project overview

A menu-driven Python application designed to help a small retailer organise product records, update stock information, search inventory, and generate basic inventory reports.

| Core capabilities                       | Business purpose                               |
|-----------------------------------------|------------------------------------------------|
| Add, update, and delete products        | Maintain structured product records            |
| Search by product ID or name            | Retrieve product information more conveniently |
| Inventory summary and low-stock listing | Support routine stock monitoring               |
| JSON file persistence                   | Retain saved records between program sessions  |

## 1. Executive Summary

Small retailers need accurate records of product names, prices, and available quantities. Manual records can be time-consuming to maintain and difficult to search. This project provides a simple Python application to manage product records and generate basic inventory summaries. JSON file handling is used to preserve records between sessions.

## 2. Business Background

**Illustrative scenario:** A small grocery retailer sells items such as milk, bread, flour, cooking oil, and household essentials. As the product catalogue grows, the retailer needs a consistent way to record stock, find products, and review quantities.

## 3. Business Problem

- **Manual record maintenance:** Updates can take time and introduce errors.
- **Limited stock visibility:** Low-stock items may be harder to identify quickly.
- **Slow product retrieval:** Searching multiple records manually is inconvenient.
- **Inconsistent records:** Changes may be missed if not recorded consistently.
- **Limited reporting:** Summarising stock and inventory value manually is cumbersome.

## 4. Business Objectives

- Organise product information in a structured format.
- Simplify product creation, modification, and deletion.
- Retrieve product information by ID or name.
- Generate inventory summaries and identify low-stock items.

- Preserve saved changes between program sessions.
- Provide a simple starting point for inventory digitisation.

## 5. Proposed Solution

The application uses Python for program logic and JSON for local persistent storage. Product records are represented as dictionaries containing an ID, name, price, and quantity, and are stored in a list. A `save_records()` function writes the inventory to `inventory.json`.

## 6. Features and Business Value

### Add products

Creates structured product records and helps maintain a central inventory list.

### Update products

Updates existing product prices and quantities when business information changes.

### Delete products

Removes discontinued or unwanted product records.

### Search products

Finds products by ID or name without reviewing every record.

### Inventory reports

Calculates product count, total quantity, estimated inventory value, and products with fewer than 10 units.

### Persistent storage

Retains saved inventory changes between sessions, provided the JSON file remains available and is written successfully.

## 7. How the System Works

- Load records from `inventory.json`; start with an empty inventory if the file is not found.
- Let the user select an operation from the main menu.
- Call the corresponding function to add, update, delete, search, or report on products.
- Save changes after add, update, and delete operations.
- Display product details or an inventory report when requested.

## 8. Business Metrics and KPIs

| KPI                     | Calculation                    | Business use                           |
|-------------------------|--------------------------------|----------------------------------------|
| Total product records   | Count of product records       | Understand catalogue size              |
| Total stock quantity    | Sum of all quantities          | Monitor recorded stock volume          |
| Total inventory value   | Sum of price $\times$ quantity | Estimate the recorded value of stock   |
| Low-stock product count | Count items below threshold    | Identify potential replenishment needs |

**Illustrative calculation:** 20 units  $\times$  ■100 per unit = ■2,000 in recorded inventory value. This is an example only, not a measured result from the project.

## 9. Expected Business Benefits

- Better organisation and accessibility of product records.
- Less reliance on manual calculations for routine inventory summaries.
- More convenient identification of recorded low-stock products.
- Continuity of saved records between program sessions.
- A foundation for future inventory and sales analytics.

Actual changes in processing time, error rates, or operating costs would need to be measured by comparing the previous process with the application in real use.

## 10. Technologies and Skills Demonstrated

**Technologies:** Python, JSON, Jupyter Notebook or another Python environment.

**Skills:** Functions, loops, conditional logic, lists, dictionaries, file handling, CRUD operations, record searching, and business-oriented reporting.

## 11. Limitations and Future Improvements

- Add validation for numeric prices and quantities.
- Track stock movements from purchases, sales, and adjustments.
- Allow configurable reorder thresholds and alerts.
- Add backups and robust file-writing error handling.
- Migrate to SQLite or MySQL for more complex data needs.
- Connect sales transactions so sales automatically reduce stock.
- Build a Power BI dashboard for stock and replenishment analysis.
- Add user access controls if multiple employees need the system.

## 12. Conclusion

This project applies Python to a practical retail operations problem. It organises product records, supports basic inventory operations, and generates summary reports. It demonstrates how a small digital solution can improve access to operational information and provides a foundation for future enhancements such as transaction tracking, database integration, and sales analytics.

---

**Project:** Project 05 — Inventory Management System

**Category:** Python Application Development

**Primary use case:** Small retail inventory management

**Status:** Core features implemented; enhancements identified for future development.